Document Number	**SAD-CMPE-131 Sec xx**
Version	**1.0**
Status	**Draft**
Created By	**Christopher Nguyen**

OurSJSU

System Architecture Document

Reviewers

Department	**Name/Title**
Engineering	Evan Ha
System Architecture	Christopher Nguyen
Product Management	Taha Khan

Team Member Contribution Record
Project Team: After Six
Deliverable: System Architecture Document for OurSJSU
Each team member must participate in the development of this deliverable. As a team, review and agree on each member's contribution level and complete the table below before submission.

Team Member	Contribution Level	Brief Description of Contribution
Evan Ha	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 2.2, 8, and 9
Christopher Nguyen	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 3
Rico Li	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 6 and 7
Taha Khan	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 1 and 2
Kyle Ringor	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 5
	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	

Contribution Levels
Full Contribution (100%) – Fully participated and contributed to the deliverable; receives 100% of the team score.
Partial Contribution (50%) – Partially participated or contributed to the deliverable; receives 50% of the team score.
No Contribution (0%) – Did not participate or contribute to the deliverable; receives 0% of the team score.

Team Confirmation:
By submitting this document, the team confirms that the contribution levels reported above have been reviewed and agreed upon by the team members.

Table of Contents

1 Introduction

1.1 Purpose of the Document

1.2 Scope

1.3 Assumptions and Dependencies

2 Functional Overview

2.1 System Context

3

List of Tables

Table 1.1:
Table 1.2:

1 Introduction

1.1 Purpose of the Document Taha Khan

- Define the functional behavior and structure of the system.
- Serve as a reference for developers, testers, and architects.

1.2 Scope

- Summarize the functions described in the Product Requirements Document (PRD).

- Provide a high-level overview of what the system will achieve.

1.3 Assumptions and Dependencies

- List assumptions made during the functional design (e.g., hardware/software availability).
- Define dependencies on external systems or components.

2 Functional Overview Taha Khan

2.1 System Context

- Provide a **context diagram** showing the system's interactions with external entities (users, external systems, etc.).
- Briefly describe how the system integrates into the overall environment.

2.2 Functional Modules Evan

- Major functional modules of the OurSJSU system:
    - **Module 1: User Authentication**
        - Purpose: Verify the student identities through SJSU Single Sign-on (SSO) and manage authenticated user sessions.
    - **Module 2: Course Data Ingestion**
        - Purpose: Collect course descriptions, prerequisites, class sections, schedules, and seat-availability information from SJSU resources and store information into the database.
    - **Module 3: Prerequisite Evaluation**
        - Purpose: Compare a student's completed and in-progress courses with course prerequisites to determine whether the student is eligible to take a course.
    - **Module 4: Degree Roadmap and Course Recommendations**
        - Purpose: Display a student's degree requirements and recommend future courses based on completed courses, prerequisites, major requirements, and academic process.
    - **Module 5: Course Search and Comparison**

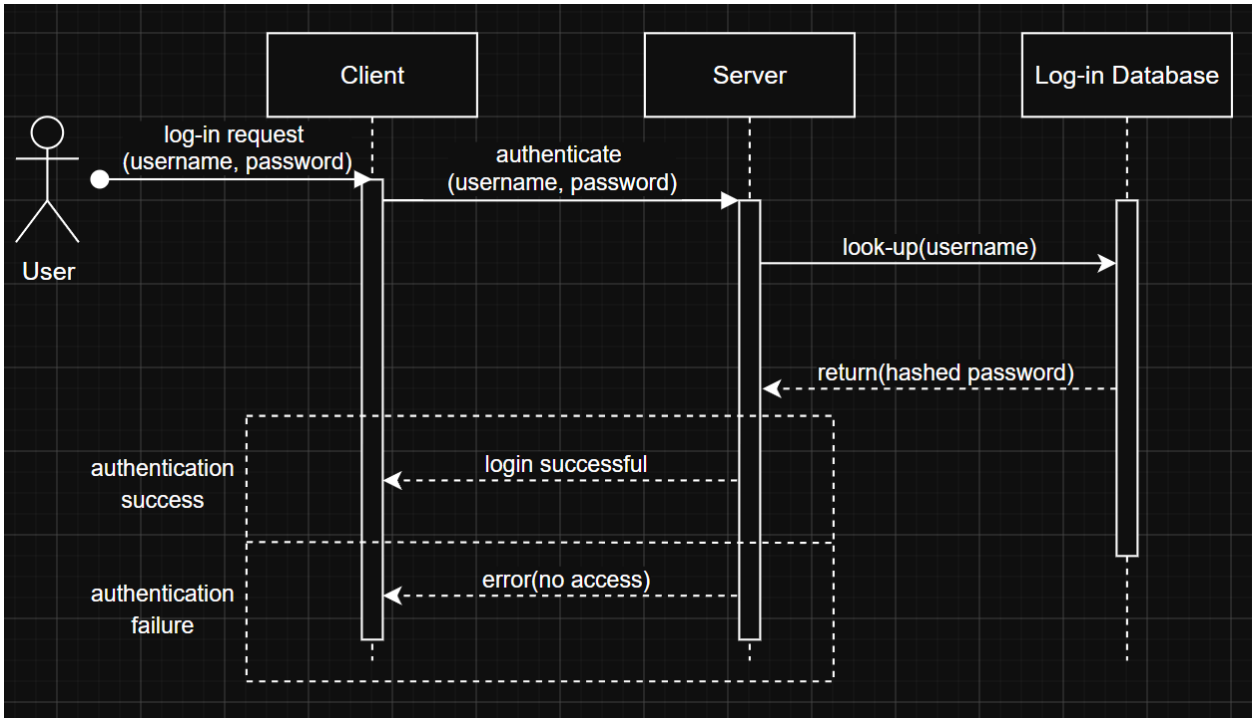
        - Purpose: Allow students to search for courses and compare available sections by instructor, meeting time, and seat availability.

- **Module 6: Course Watchlist and Availability Monitoring**
    - Purpose: Allow students to save courses or sections to a watchlist and monitor those sections for changes in seat availability.
- **Module 7: Notification Style**
    - Purpose: Send alerts through email, SMS, Discord, or Slack when the availability of a watched section changes.
- **Module 8: External Ratings Integration**
    - Purpose: Provide links to RateMyProfessors and MyClassGrades so students can view instructor and grade information.
- **Module 9: MySJSU Redirection**
    - Purpose: Redirect students to the official MySJSU portal to complete course enrollment, which remains outside the scope of OurSJSU.

3 System Models

3.1 Sequence Diagrams

Use Case Name: Login process
Actors: User, Application
Preconditions: The user has entered their username/email and password into the login screen and submitted a login request with their credentials entered
Postconditions: If the user has entered a valid pair of credentials, the login is successful and the rest of the application will be accessible. If the user did not enter a valid pair of credentials, the login will fail and the user will remain on the login screen
Flow of Events: After the user enters their credentials and submits their login request through their client, the client forwards their data to the server which does a lookup in the log-in credentials database if the user entered a registered account. If the user's information is in the database, the server authorizes the client to enter the application. If the user's information is not in the database, the login attempt fails and the user will remain on the login screen.



3.2 State Diagrams

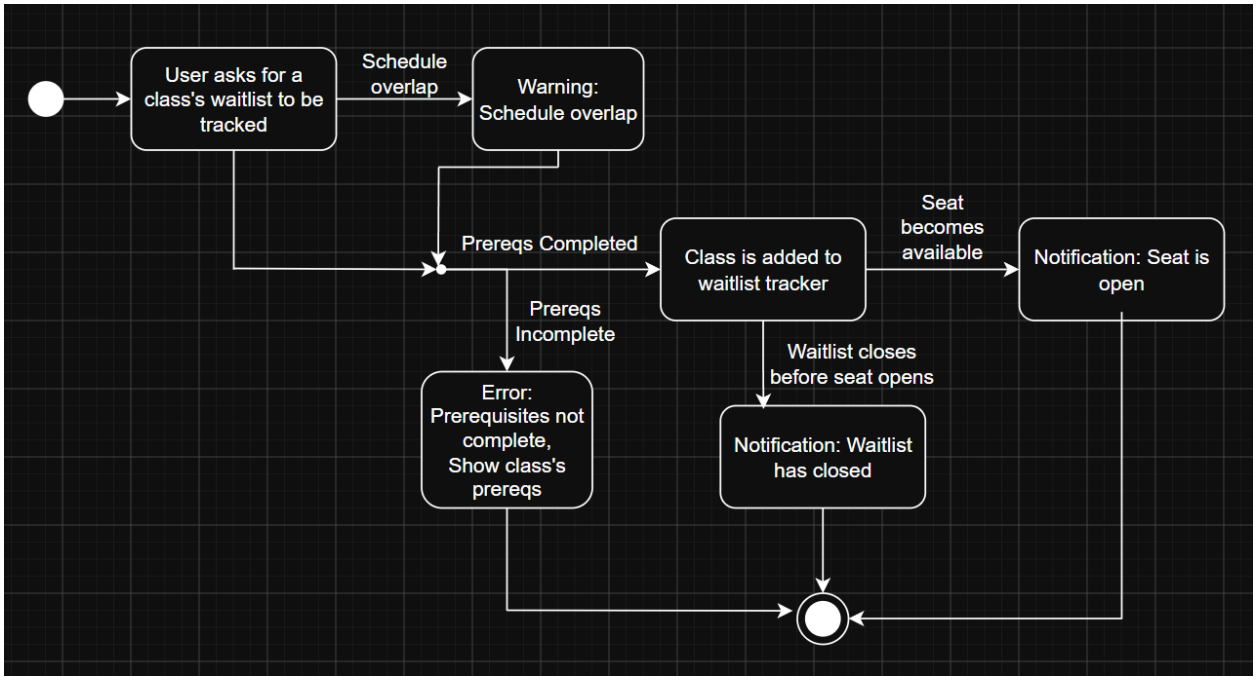
Use Case Name: Waitlist track

Actors: User, Application

Preconditions: The user attempts to track a class's waitlist

Postconditions: If the user has not completed or is currently enrolled in the class's prerequisites, the system tells the user that they will not be able to take the class. If the user has cleared the prerequisites and a seat becomes available, the system will send the user a notification about the opening. If the waitlist closes before any opening appears, the system sends the user a notification about the waitlist ending.

Flow of Events: After the user asks to track the class, if the class takes place at the same time as another class they are currently signed up for, the system gives them a warning, but still proceeds in case the user is planning to change out the overlapping class if the new one opens. Next, if the user has not completed or is not currently enrolled in all of the prerequisite classes for the tracked class, the system gives them an error message that they will not be able to take the class. Otherwise, the class is added to their waitlist tracker. If a seat opens, the system sends a notification to the user through their preferred method of communication telling them of the opening. If the waitlist closes without any opening, the system sends the user a notification that the waitlist has ended.



4 Architectural Design MKH

4.1 System Architecture

- Provide a high-level **architecture diagram** that shows:
  - Major system components/modules.
  - Communication paths between components.
- Example: Client-server architecture, layered architecture, etc.

Whem

4.2 Component Descriptions

- Describe each architectural component, its purpose, and its interaction with other components. Example:

Component 1: Web Interface

- Purpose: Provide a graphical user interface for users.
- Dependencies: Requires an API layer to fetch data.

5 Functional Specifications

5.1 Detailed Functional Requirements

- Provide detailed descriptions of each system function, building on the PRD:
  - **FR1**: Authenticates users via SJSU Single Sign-On

        - **Inputs**:
            - Username
            - Password
        - **Outputs**:
            - Session token (user_id, role, expiration_timestamp)
            - User profile object (student_id, full_name, email, role).
        - **Dependencies**:
            - Database (User Profile table)
            - SJSU SSO Provider
    - **FR2**: Course Catalog & Class Schedule Data
        - **Inputs**:
            - SJSU Term identifier (fall-2026).
            - Target academic subject (CMPE or EE)
            - Raw HTML page (https://www.sjsu.edu/classes/schedules/fall-2026.php)
        - **Outputs**:
            - IngestionDocument (Section records, CatalogCourse records)
            - Database Records (Courses, Sections, and Prerequisite Rules tables)
        - **Dependencies**:
            - Database
            - Ingestion Scraper
    - **FR3**: Prerequisite Evaluation Engine
        - **Inputs**:
            - course_id (CMPE-130)
            - StudentContext: List of CourseAttempt (course key, status: completed | in_progress, grade: string), earned_units (int), gpa (float), standing (string), major (string)
        - **Outputs**:
            - EligibilityResult: Overall status (eligible, ineligible, unknown)
        - **Dependencies**:
            - Database (CatalogCourse, PrerequisiteRule)
    - **FR4**: Visual Degree Roadmap & Course Recommendation
        - **Inputs**:
            - student_id (UUID).
            - major_code (e.g., CMPE_BS).
        - **Outputs**:
            - Degree roadmap graph
            - Recommended course list
        - **Dependencies**:
            - Database (Major Roadmaps, Student History)
            - Prerequisite Evaluation Engine
    - **FR5**: Course Watchlist & Seat Availability Monitor

        - **Inputs**:
            - Student identifier.
            - Selected term and course or section identifiers.
            - Updated seat availability records.
        - **Outputs**:
            - Updated watchlist after additions or removals.
            - Availability-change events for watched sections.
        - **Dependencies**:
            - Watchlist and section database.
            - Ingestion scraper and availability monitor.
    - **FR6**: Notifications
        - **Inputs**:
            - Availability-change event.
            - Student notification preferences and delivery destination.
        - **Outputs**:
            - Email
            - SMS
            - Discord
            - Slack alert.
        - **Dependencies**:
            - Notification providers.
    - **FR7**: External Ratings
        - **Inputs**:
            - Instructor name or course identifier.
        - **Outputs**:
            - Relevant RateMyProfessors and MyClassGrades links.
        - **Dependencies**:
            - External rating websites and verified link mappings.

5.2 Interface Specifications

- Define how modules and components will interact with each other:
    - **Internal Interfaces**:
        - **Frontend to Backend**: Exchange requests and responses for profiles, academic progress, roadmaps, search, watchlists, and preferences. The backend validates requests and enforces access permissions.
        - **Backend to Database**: Retrieve course information and store student progress, watchlists, and notification history.
        - **Ingestion Scraper to Database**: Import validated catalog and schedule records with update timestamps.
        - **Backend to Prerequisite Engine**: Submit student history and course requirements; receive eligibility results.

        - **Availability Monitor to Notification Service**: Send detected availability changes and subscriber information for alert delivery.
    - **External Interfaces**:
        - **SJSU SSO**: Verify user identity through an approved integration; access and protocol require confirmation.
        - **SJSU Catalog & Class Schedule**: Supply course, prerequisite, section, and availability data.
        - **Notification Providers**: Deliver alerts through email, SMS, Discord, and Slack interfaces.
        - **RateMyProfessors & MyClassGrades**: Provide external links for supplemental information.
        - **MySJSU**: Redirect students to official enrollment. Enrollment remains outside OurSJSU.

6 System Behavior

6.1 Event Handling

The OurSJSU system reacts to user actions, scheduled background processes, and error conditions

User Authentication Request:

Upon receiving user credentials, the client forwards them via HTTPS to the Backend. The Backend delegates verification to the SJSU Single Sign-On (SSO) provider. If verified, a session token containing the ‘user_id’, ‘role’, and expiration timestamp is generated and returned. If verification fails, the system logs the attempt, denies access, and returns an error message to the user UI.

Course Search and Filter:

When a student enters search criteria (e.g., subject, instructor, time), the Frontend sends a request to the Backend. The Backend queries the Database and returns matching course sections along with links to external rating services (RateMtProfessors/MyClassGrades).

Prerequisite and eligibility Check:

When a user views a course, the Frontend requests an evaluation. And the Backend feeds the student’s ‘StudentContext’ (Completed/in-progress courses, GPA, units) and target ‘course_id’ into the Prerequisite Evaluation Engine. The engine returns an ‘Eligibility Result’ (eligible, ineligible, or unknown).

Watchlist addition:

When a user checks a section on their watchlist, the Backend updates the section record and triggers an ‘Availability_Change’.

Aviability Updates:

The Scraper periodically parses HTML from SJSU Schedules. If a section’s status changes, like from close to open, the Database updates the section record and trigger the ‘Availability_Change’.

Notification Event:

When ‘Availability_Change’ active, the monitor identifies all subscribes students from the Database. It passes the aldert details to the Notification Dispatch API. and transmit messages through the students’ selected channels like Emails, SMS and Discord.

External Redirection:

‘Enroll’ button triggers an outbound redirect event, navigating the user to the official MYSJSU portal without processing any enrollment logic within OurSJSU.

Scraper Exception:

If the SJSU Schedule is unreadable or changes structure, the Scraper logs an alert to the system sudit log, retry using exponential backoff, and serves th last cached catalog data to users.

6.2 Performance Constraints

Login Response Time:

SSO authentication and session creation must complete within 2 second under normal operating loads.

Catalog and Aviability Update Latency:

Data ingestion scrapers must run cycles frequently enough to detect and reflect seat availability changes within 5 minutes of being published on SJSU sources.

Prerequisite Evaluation Latency:

Prerequisite checks and degree roadmap generation must return results to the UI in 1 sec.

Notification Delivery Time:

Once an availability change event is generated, notification dis patches via SMS,Email or Discord must initiate within 30 sec.

7 Non-Functional Specifications

7.1 Performance Requirements

Data Processing:

The web scraping and schedule ingestion pipeline must complete a full check cycle across all offered course section within 5 minutes.

UI load speed:

Dynamic views must render in 2 secs on standard broadband connections.

Concurrent Capacity:

The application and database infrastructure must sustain concurrent queries and active monitoring for thousands of active SJSU student profiles during peak registration periods.

7.2 Security Requirements

Authentication:

User identity verification must be handled exclusively via SJSU SSO protocols. No plain-text passwords or primary SJSU password credentials shall be stored or logged within OurSJSU databases.

Session Management:

Authenticated sessions must be maintained using secure, encrypted session tokens with enforced idle timeouts and role-based access controls.

Data Protection & Privacy:

Student academic records must be restricted so that students can only access their own private profile data. All client-to-server and backend-to-external API communication must be encrypted using HTTPS.

Audit Logging:

The system must maintain secure audit logs tracking all user sign-in attempts, administrative actions, and failed authorization requests.

7.3 Reliability Requirements

Notification Delivery:

The notification service must achieve a successful delivery rate for seat aviability alert across supported delivery channels.

Scraping Resilience:

Fauliures in web scraping cycles or third-party integrations must not crash core application services. The system should also degrade gracefully by serving cached data.

System Abailability:

The core platform must target uptime during active registration windows.

8 System Models for Validation Evan

- Provide models specifically for validating that the system meets functional and non-functional requirements:

Simulation models:

The following simulation models represent important system processes under different input conditions.

Model	Inputs or Event	Expected System Response
User Authentication	Student submits valid SJSU credentials.	The system verifies the student and creates an authenticated session.
User Authentication	Student submits invalid credentials.	The system rejects the login and displays an error message.
Prerequisite Evaluation	Student has completed all prerequisites for a selected course.	The system marks the student as eligible for the course.

Prerequisite Evaluation	Student is missing one or more prerequisites.	The system marks the student as ineligible and identifies the missing requirements.
Course Recommendation	Student submits their major and course history.	The system generates a degree roadmap and recommends eligible future courses.
Availability Monitoring	A watched section changes from closed to open.	The system creates an availability-change event.
Notification Delivery	The system creates an availability-change event.	The system sends an alert through the student's preferred notification method.
External Ratings	Student selects an instructor or course.	The system provides relevant RateMyProfessors and MyClassGrades links.

Test scenarios derived from use case models:

Functional Test Scenarios:

- **Test Scenario 1: Successful Student Login**
  - Related requirement: FR1 – User Authentication
  - Precondition: The student has a valid SJSU account.
  - Test Steps:
    i. Open the OurSJSU login page
    ii. Select the SJSU Single Sign-on (SSO) option
    iii. Enter valid SJSU credentials
    iv. Submit login request
  - Expected Result: The student is authenticated and redirected to their profile or dashboard.

- **Test Scenario 2: Failed Student Login**
  - Related requirement: FR1 – User Authentication
  - Precondition: The student enters invalid credentials.
  - Test Steps:
    i. Open OurSJSU login page
    ii. Enter invalid login credentials
    iii. Submit login request
  - Expected Result: Access is denied, no session is created, and an error message is displayed.

- **Test Scenario 3: Prerequisite Evaluation**
  - Related requirement: FR3 – Prerequisite Evaluation Engine

- Precondition: The student's course history is stored in the system.
- Test Steps:
  i. Search for a course with prerequisites
  ii. Select the course
  iii. Request an eligibility check
- Expected Result: The system compares the prerequisites with the student's history and return eligible, ineligible, or unknown.

- **Test Scenario 4: Degree Roadmap and Recommendations**
  - Related requirement: FR4 – Degree Roadmap and Course Recommendation
  - Precondition: The student's major and completed courses are stored.
  - Test Steps:
    i. Open the degree-roadmap page
    ii. Request course recommendations
  - Expected Result: The system displays the student's degree progress and recommends eligible future courses.

- **Test Scenario 5: Course Search and Comparison**
  - Related Requirement: Course Search and Comparison
  - Precondition: Current course and section information is available.
  - Test Steps:
    i. Search for a course
    ii. Select multiple available sections
    iii. Open the comparison view
  - Expected Result: The system displays the sections side by side with their instructors, schedules, and availability.

- **Test Scenario 6: Course Watchlist**
  - Related Requirement: FR5 – Course Watchlist and Seat Availability Monitor
  - Precondition: The student is logged in.
  - Test Steps:
    i. Search for a course or section
    ii. Add it to the watchlist
    iii. Open the student's watchlist
  - Expected Result: The selected course or section appears in the student's watchlist.

- **Test Scenario 7: Availability Notification**
  - Related Requirement: FR5 and FR6
  - Precondition: A closed section is on the student's watchlist.
  - Test Steps:
    i. Simulate the watched section changing from closed to open
    ii. Allow the availability monitor to process the update

- Expected Result: The system detects the change and sends an alert through email, SMS, Discord, or Slack.

- **Test Scenario 8: External Rating Access**
  - Related Requirement: FR7 – External Rating
  - Precondition: A valid external rating link exists.
  - Test Steps:
    i. Select an instructor or course
    ii. Select the external-rating option
  - Expected Result: The system provides the relevant RateMyProfessors or MyClassGrades link.

- **Test Scenario 9: MySJSU Redirection**
  - Related Requirement: MySJSU External Interface
  - Precondition: The student has selected a course.
  - Test Steps:
    i. Select the enrollment option
    ii. Follow the provided MySJSU link
  - Expected Result: The student is redirected to the official MySJSU portal. OurSJSU does not process the enrollment.

Non-Functional Validation Scenarios:

Requirement	**Validation Method**	**Passing Condition**
Performance	Measure the course-data update time after an availability change.	Updated information appears within five minutes.
Usability	Ask test users to complete common tasks such as searching for and watching a course.	Users complete the tasks without significant confusion or assistance.
Security	Attempt to access protected student information without authentication.	The system denies access and does not expose student information.
Capacity	Simulate thousands of student profiles and concurrent database requests.	The system processes requests without failure or data loss.
Reliability	Simulate multiple notification events and record successful deliveries.	At least 99% of notifications are delivered successfully.

9 Appendices Evan

- Detailed diagrams: no additional diagrams included in current version

Glossary:

Term	Definition
API	A method that allows different software systems to communicate with each other.
Availability Monitor	The component that checks watched class sections for seat-availability changes.
Backend	The part of OurSJSU that processes requests and communicates with the database and other components.
CatalogCourse	A database record containing course information collected from the SJSU Course Catalog.
CourseAttempt	A record of a course that a student has completed or is currently taking.
Database	The system used to store course information, student profiles, academic progress, watchlists, and notification history.
Degree Roadmap	A visual presentation showing degree requirements, courses, and prerequisite relationships.
Eligibility Result	The result of a prerequisite evaluation.
External Ratings	Instructor and grade information accessed through RateMyProfessors and MyClassGrades.
Frontend	The part of OurSJSU that students directly interact with.
Functional Requirement	A specific function or capability that OurSJSU must provide.
IngestionDocument	A collection of course and section records created during the data-ingestion process.
Ingestion Scraper	The component that collects course catalog, class schedule, and seat-availability information from SJSU resources.
Model-View-Controller (MVC)	The architectural pattern used to separate the system's data, user interface, and control logic.
MyClassGrades	An external service that provides past grade information.
MySJSU	The official SJSU student portal.

Notification Provider	A service that delivers alerts through email, SMS, Discord, or Slack.
OurSJSU	The proposed SJSU course-planning system.
Prerequisite	A requirement that must be completed before a course.
Prerequisite Evaluation Engine	The component that compares student information with course requirements to determine eligibility.
RateMyProfessors	An external service that provides instructor ratings.
Session Token	Information created after a successful login that identifies the user and the session's expiration time.
Single Sign-On (SSO)	SJSU authentication used for login.
StudentContext	Student information used during prerequisite evaluation, including course history, earned units, GPA, standing, and major.
User Profile	A stored record containing a student's ID, name, email, and role.
UUID	A unique value used for identifying a student record.
Watchlist	A saved list of courses or sections that a student wants OurSJSU to monitor.
Web Scraper	A component that automatically collects information from SJSU web pages.

- **Meeting Minutes:**
  - Attendees:
  - Location:
  - Discussion:
  - AI/PICs: